南京信息工程大学　试卷

2020－2021 学年第 2 学期 数据结构 课程试卷（期末 A 卷）

本试卷共 2 页；考试时间 120 分钟；任课教师 陈胜 ；出卷时间 2021 年 6 月

（闭卷考试，C 语言版）

学院 _____ 专业 _____ 班

学号 _____ 姓名 _____ 得分 _____

一、数据结构题（共 25 分）

1、说明顺序表与单链表的优缺点，举一个采用单链表存储数据更为合理的例子。（10 分）

2、写出数据元素为 int 型的（单向）链式栈存储结构代码。（5 分）

3、图 2 是图 1 中矩阵的存储示意图。写出相应存储结构代码。（10 分）

$$\begin{bmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -8 \end{bmatrix}$$

图 1　矩阵

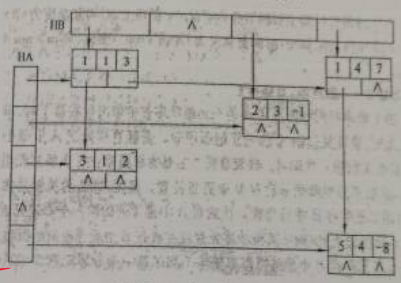


图 2　存储示意图

十字链表

二、算法基础题（共 40 分）

1、画一棵结点为 ABCDEF 且前序遍历结果为 ABCDEF 的完全二叉树。（4 分）

2、画一棵结点权重分别为 2、4、6、8、10、12 的哈夫曼树。（4 分）

3、画出图 3 中图的一棵以 v_0 为树根的最小生成树。（4 分）

4、给出图 4 中活动 a_4 的最早开始时间与最迟开始时间。（4 分）

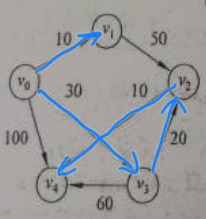


图 3　带权有向图

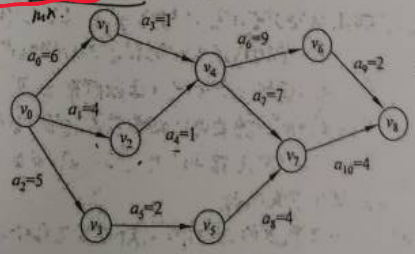


图 4　AOE 网

5、给出模式匹配 KMP 算法中模式为“bdabda”时的 next 数组中各元素的值。（4 分）

6、谈谈最短路 Dijkstra 算法的基本思想，并说明该算法的正确性。（5 分）

7、谈谈折半查找算法背后的核心思想，并介绍该算法时间复杂度。（5 分）

8、分别给出基于大根堆和二叉排序树的排序算法，并比较这两种算法的优劣。（10 分）

三、 综合能力题（共 35 分）

1、现欲设计一个算法，用来查找一个有向图中可能存在的有向回路，并且在存在有向回路的情况下，输出其中一个有向回路。用文字配合必要的代码，介绍自己所设计的算法的基本思想，并以图 3 中的有向图为例，画出该算法中所采用的图存储结构的示意图。（10 分）

2、图 5 中是二叉树层序遍历算法的部分代码。阅读此部分代码后，完成下面问题：

（1）在图 5 中下划线处填写代码，使得填写后的代码可实现该遍历算法的功能；（10 分）

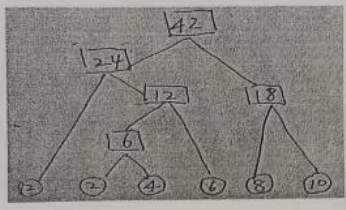
（2）写出图 5 代码中 BiNode 的存储结构代码；（5 分）

（3）当队列 Q 非空时，队列 Q 中的指针总是指向哪些的树结点？（5 分）

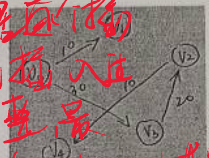
（4）说明该遍历算法可以采用队列来实现的原因。（5 分）

```
01  void levelOrder(BiNode *root)
02  {
03      BiNode *q = NULL, *Q[MAXSIZE];
04      int front = ____, rear = ____;
05      if (root == NULL) ____;
06      Q[++rear] = root;
07      while (____)
08      {
09          q = ____;
10          printf("%c", q->data);
11          if (q->lchild != NULL) Q[++rear] = q->lchild;
12          if (q->rchild != NULL) Q[++rear] = q->rchild;
13      }
14  }
```

图 5　二叉树层序遍历算法部分代码



3、参考答案（4 分）：



4、关键路径长度为 18；活动 a_4 最早开始时间为 4（2 分）；最迟开始时间为 6（2 分）。

5、参考答案（4 分）：-1,0,0,0,1,2。

6、Dijkstra 算法基本思想（3 分）；算法正确性（2 分）。

7、折半查找算法核心思路是：二分法（3 分）；算法复杂法：$O(\log_2 n)$（2 分）。

8、基于大根堆排序算法简介（4 分）；基于排序树排序算法简介（4 分）；优劣说明的合理性（2 分）。

三、综合能力题

1、（1）应画示意图可以用来描述图 3 中有图的存储结构（3 分），并且，该示意图结出的数据结构可用于所提的算法（2 分）。

（2）所给算法基本思想的描述清晰（2 分），能实现所需的功能（3 分）。

2、

（1） 参考答案：

第 04 行代码（2 分）：rear = -1

第 05 行代码（2 分）：return

第 07 行代码（2 分）：front != rear

第 09 行代码（2 分）：q = Q[++front]

第 11 行代码（2 分）：q->lchild != NULL

（2）参考答案（5 分）

```
typedef char DataType;
typedef struct BiNode;
{
    DataType data;
    struct BiNode *lchild, *rchild;
}
```

（3）参考答案：对于队首元素是树根结构的情况，即刚初始化后，队列 Q 中只有一个指针，指向树根结点（1 分）。对于队首元素不是根结构的情况下，队列 Q 中指针指向的是：队首元素在树中的同层右侧所有元素和自己（2 分），以及队首元素在树中的同层左侧所有元素的孩子（2 分）。

（4）参考答案：按树层序遍历算法基本思路，任意一个结点，比其同层中（按从左到右规则排列的）后面的结点先输出（1 分），并且，其左右孩子结点也比那些后面结点的左右孩子先输出（2 分），从而可利用队列先进先出的特点来实现此算法，也即，输出队伍队头结点时，同时在队尾加入该结点的左右孩子结点（2 分）。

南京信息工程大学　试卷

2019－2020 学年第 2 学期数据结构与算法基础课程试卷（A 卷）

本试卷共 2 页；考试时间 120 分钟；任课教师 陈胜 ；出卷时间 2020 年 6 月

（闭卷考试，C 语言版）

学院 _____ 专业 _____ 班

学号 _____ 姓名 _____ 得分 _____

一、数据结构题（共 25 分）

1、说明顺序表与单链表的优缺点，并举一个采用单链表存储数据更合理的例子。（10 分）

2、写出数据元素为 char 型的（单向）链队列存储结构定义的代码。（5 分）

3、图 2 是图 1 中矩阵的存储示意图。写出相应存储结构定义的代码。（10 分）

$$\begin{bmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & -1 & 0 \\ 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -8 \end{bmatrix}$$

图 1　矩阵

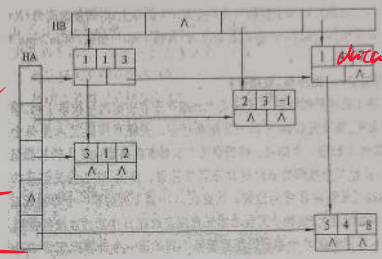


图 2　存储示意图

二、算法基础题（共 35 分）

1、画一棵结点为 A、B、C、D、E 并且前序遍历结果为 ABCDE 的完全二叉树。（5 分）

2、画一棵结点权重分别为 1、2、3、4、5、6 的哈夫曼树。（5 分）

3、给出图 3 中结点 v_0 到结点 v_4 的最短路径。（5 分）

4、给出图 4 中活动 a_6 的最早开始时间与最迟开始时间。（5 分）

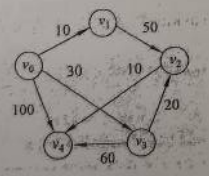


图 3　带权有向图

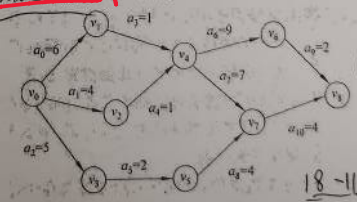


图 4　AOV 网

5、说明可以采用栈来实现图深度优先遍历算法的原因。（5 分）

6、给出可采用递归方法实现折半查找算法的原因，并介绍该算法的时间复杂度。（5 分）

7、介绍起泡排序算法的内容，并说明该算法是一个有效的排序方法。（5 分）

三、综合能力题（共 40 分）

1、现欲设计一个算法，求输出任意给定的一个简单有向图中所有的简单有向回路。用文字配合必要的代码，简要地给出所设计的算法，并且，以图 3 中的图为例，画出算法中所采用图存储结构的示意图。（10 分）

2、图 5 中是求最小生成树的 Prim 算法部分代码。阅读图中代码后，完成下面问题：

（1）填写图中下划线处缺失的代码；（10 分）

（2）用代码写出图中 MGraph 的定义；（5 分）

（3）写出图中第 13 行代码处 MinEdge 函数的实现代码；（5 分）

（4）证明 Prim 算法所生成的树是最小生成树。（10 分）

```
01  void Prim(MGraph *G, int v)                          /*假设从顶点 v 出发*/
02  {
03      int i, j, k;
04      int adjvex[MaxSize], lowcost[MaxSize];
05      for (i = 0; i < G->vertexNum; i++)               /*初始化辅助数组*/
06      {
07          lowcost[i] = G->edge[v][i];
08          adjvex[i] = v;
09      }
10      lowcost[v] = 0;                                  /*将顶点 v 加入集合 U*/
11      for (k = 1; k < G->vertexNum; k++)               /*迭代 n-1 次*/
12      {
13          j = MinEdge(lowcost, G->vertexNum);          /*寻找最短边的邻接点 j*/
14          printf("(%d, %d)%d ", adjvex[j], j, lowcost[j]);
15          lowcost[j] = 0;                              /*将顶点 j 加入集合 U*/
16          for (i = 0; i < G->vertexNum; i++)           /*调整辅助数组*/
17              if (G->edge[j][i] < lowcost[i])
18              {
19                  lowcost[i] = G->edge[j][i];
20                  adjvex[i] = j;
21              }
22      }
23  }
```

图 5　Prim 算法部分代码

南京信息工程大学

(19)_2020学年 第 2 学期 数据结构课程 期末 A 卷

试卷共 4 页；考试时间 120 分钟；任课教师 陈胜 ；出卷时间 2020 年 6 月

一、 数据结构题

1、对存储线性表所需空间（2 分）、访问元素的时间复杂性（2 分）、增删元素的时间复杂性与空间复杂性（2 分）这三个角度来辩证地论述两者之类的优缺点；所举例子，清晰、相关（2 分），并且理由合理（2 分）。

2、所给代码的链结点中数据是 int 型元素（1 分）；所给代码的链结点含有指下一个结点的指针（2 分），和指向栈顶的指针（2 分）。参考答案：

```
typedef int DataType;
typedef struct Node
{
    DataType data;
    sturct Node *next;
} *top;
```

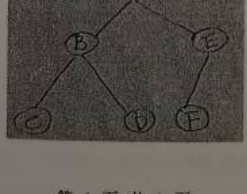
3、十字链表结点结构体中元素（4 分），十字链表结构体中元素（4 分），其他 2 分。

参考答案：

```
typedef int DataType;
typedef struct OrthNode
{
    int row_NO, col_NO;
    DataType data;
    OrthNode *down, *right;
}
typedef struct SparseMatrix
{
    struct OrthNode *HA[MaxSize],*HB[MaxSize];
    int row_Length, col_ Length;
} *sparseMatrix;
```

二、 算法基础题

1、参考答案（4 分）：



```
    {   if (b->data==x) p=NULL;
        else if (b->lchild!=NULL && b->lchild->data==x)
                p=b;
        else if (b->rchild!=NULL && b->rchild->data==x)
                p=b;
        else
        {   findparent(b->lchild,x,p);
            if (p==NULL)
                findparent(b->rchild,x,p);
        }
    }
    else p=NULL;
}
```

2. （10 分）解：算法如下：

```
int visited[MAXV];
void DFS(ALGraph *G,int v)                  //深度优先遍历算法
{   ArcNode *p;
    visited[v]=1;                           //置已访问标记
    p=G->adjlist[v].firstarc;               //p指向顶点v的第一个邻接点
    while (p!=NULL)
    {   if (visited[p->adjvex]==0)          //若p->adjvex顶点未访问,递归访问它
            DFS(G,p->adjvex);
        p=p->nextarc;                       //p指向顶点v的下一个邻接点
    }
}
bool DFSTrave(ALGraph *G,int i,int j)
{   int k;
    bool flag1=false,flag2=false;
    for (k=0;k<G->n;k++)
        visited[k]=0;
    DFS(G,i);                   //从顶点i开始进行深度优先遍历
    if (visited[j]==1)
        flag1=true;
    for (k=0;k<G->n;k++)
        visited[k]=0;
    DFS(G,j);                   //从顶点j开始进行深度优先遍历
    if (visited[i]==1)
        flag2=true;
    if (flag1 && flage2)
        return true;
    else
        return false;
}
```

3.（15 分）（1）采用快速排序的算法如下：（12 分）

```
int QuickSelect(int R[],int s,int t,int k)    //在R[s..t]序列中找第k小的元素
```

```
{    int i=s,j=t;
     int tmp;
     if (s<t)                              //区段内至少存在2个元素的情况
     {    tmp=R[s];                        //用区段的第1个记录作为基准
          while (i!=j)                     //从区段两端交替向中间扫描,直至i=j为止
          {    while (j>i && R[j]>=tmp)
                    j--;                   //从右向左扫描,找第1个小于tmp的R[j]
               R[i]=R[j];          //将R[j]前移到R[i]的位置
               while (i<j && R[i]<=tmp)
                    i++;                   //从左向右扫描,找第1个大于tmp的R[i]
               R[j]=R[i];                  //将R[i]后移到R[j]的位置
          }
          R[i]=tmp;
          if (k-1==i) return R[i];
          else if (k-1<i) return QuickSelect(R,s,i-1,k);   //在左区段中递归查找
          else return QuickSelect(R,i+1,t,k);              //在右区段中递归查找
     }
     else if (s==t && s==k-1)     //区段内只有一个元素且为R[k-1]
          return R[k-1];
}
void Mink(int R[],int n,int k)   //输出整数数组R[0..n-1]中第k小的元素
{    if (k>=1 && k<=n)
          printf("%d\n", QuickSelect( R,0,n-1,k));
}
```

（2）对于求 R 中第 k 小元素的算法 Mink(R，n，k)，设其算法平均执行时间为 $T(n)$，有以下递推式：（3 分）

$T(1)=1$

$T(n)=T(n/2)+O(n)$

则：$T(n)= T(n/2)+O(n)= T(n/2^2)+O(n)+O(n/2)=\cdots$

$=T(n/2^m)+O(n)+O(n/2)+\cdots+O(n/2^m)$　　//$m=\log_2 n$

$=O(1)+O(n)+O(n)$

$=O(n)$

所以，该算法的平均时间复杂度为 $O(n)$。